

Name: SARANSH MITTAL
SRN: PES1UG24AM254

Interfaces:

Interface	Purpose
I1: CourierRequest	Sender submits pickup and destination details to the dispatch service.
I2: RiderAssignment	Dispatch service sends rider assignment and pickup information to the rider app.
I3: RiderTelemetry	Rider app transmits live GPS position and delivery status updates.
I4: RouteOTPREquest	Dispatch/routing service requests route optimization and destination OTP verification.
I5: CourierData	Service components store and retrieve courier, rider, route and OTP data.
I6: Notifications	Dispatch service sends delivery and status notifications to the sender client.

Architecture Analysis:

Architecture	Advantages	Limitations	Suitability
Layered Architecture	Clear separation of presentation, business logic and data layers.	Tightly coupled layers make independent updates difficult.	Suitable for simple systems with clear functional boundaries.
Microservices Architecture	Independent services can scale separately; supports operational complexity.	More operational overhead; requires service discovery and orchestration.	Highly suitable for complex, distributed systems like dispatch and tracking.
Client–Server Architecture	Simple centralized model with straightforward communication.	Central server can become a bottleneck; less scalable.	Suitable for small-scale applications like basic tracking and dispatch.

Architecture Analysis (continued):

The Hyperlocal Courier Dispatch & Tracking Engine must coordinate rider assignment, live GPS telemetry, multi-stop route optimization, destination OTP verification and notifications. These functions have different workloads and availability requirements, making independent services a strong architectural fit.

Final Choice For Architecture: Microservices Architecture

Justification of the Choice:

- **Independent scaling:** Dispatch and live tracking can be scaled independently when courier traffic or GPS updates increase.
- **Performance:** Route optimization can run as a dedicated service without blocking rider assignment or notification processing.
- **Security:** OTP verification can be isolated as a dedicated service, limiting access to sensitive verification data and enforcing controlled service boundaries.
- **Fault isolation:** A failure in notifications or routing does not have to bring down the complete courier platform.
- **Real-time tracking:** Rider telemetry can be handled by a dedicated tracking component optimized for frequent location updates.
- **Maintainability:** Dispatch, routing, OTP and notification functionality can be developed, tested and deployed independently.
- **Extensibility:** Additional delivery features or integrations can be added as separate services without redesigning the entire system.

Key system components: Sender Client, Dispatch & Matching Service, Delivery Rider App, Order & Tracking Database, Notification Service, and Map & Routing Service. The component diagram uses UML component notation and labelled interfaces to show how these services communicate.

Component and Interface Diagram:

Component and Interface Diagram

Component and Interface Diagram:

Component and Interface Diagram

